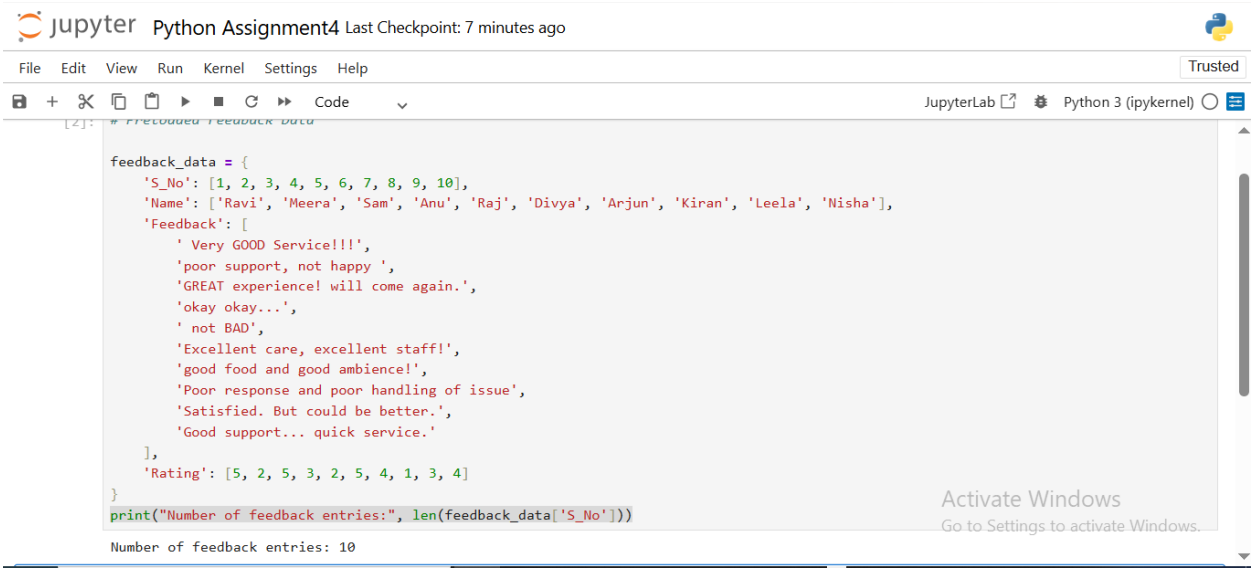
Python Fundamentals Capstone Project

Survey Feedback Analyzer

NAME: UMAYAL.K
SUBMISSION DATE: 21 - 10 - 2026

PREVIEWS:

Step 1: Preloaded Feedbacks (Given Data)



- This step creates a **preloaded dataset** called `feedback_data` that contains customer feedback information.
- Instead of asking users to enter data manually, the program starts with 10 feedback records already stored in a dictionary.

The dictionary has **4 keys**:

1. **S_No**
    - Stores serial numbers from 1 to 10.
    - Used to uniquely identify each feedback record.
2. **Name**
    - Stores the names of customers.
    - Example: Ravi, Meera, Sam, etc.

3. **Feedback**
    - Stores the comments given by customers about the service.
    - Some comments are positive, some negative, and some neutral.
    - The feedback intentionally contains extra spaces, capital letters, and special characters to practice text cleaning.
4. **Rating**
    - Stores ratings given by customers.
    - Ratings range from 1 to 5.
    - Higher ratings indicate better customer satisfaction.

Step 2: Add More Feedbacks:

```
[3]: count = int(input("How many feedbacks do you want to add? "))

for i in range(count):

    name = input("Enter customer name: ")
    feedback = input("Enter feedback: ")
    rating = int(input("Enter rating (1-5): "))

    # Generate next serial number
    new_sno = len(feedback_data['S_No']) + 1

    # Append data
    feedback_data['S_No'].append(new_sno)
    feedback_data['Name'].append(name)
    feedback_data['Feedback'].append(feedback)
    feedback_data['Rating'].append(rating)

print("Feedbacks added successfully!")
```

Activate Windows

```
How many feedbacks do you want to add?  3
Enter customer name:  Shri
Enter feedback:  Very Good Service!!!
Enter rating (1-5):  5
Enter customer name:  Mathi
Enter feedback:  Good Service!
Enter rating (1-5):  4
Enter customer name:  Zara
Enter feedback:  Average Service
Enter rating (1-5):  3
Feedbacks added successfully!
```

1.Ask the user how many feedbacks they want to add.

2.Use a loop to collect details for each new feedback.

3.For every feedback, ask the user to enter:

- Name
- Feedback text

- Rating (1 to 5)

4.Generate the Serial Number automatically.

- Existing records already have S_No from 1 to 10.
- New records should start from 11.
- Each new feedback gets the next available serial number.

5.Append the new values to the corresponding lists in the `feedback_data` dictionary.

```
print(feedback_data)
```


```
{'S_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18], 'Name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Raj', 'Divya', 'Arju
n', 'Kiran', 'Leela', 'Nisha', 'john', 'Divi', 'Mathi', 'Zara', 'Shri', 'Shri', 'Mathi', 'Zara'], 'Feedback': [' Very GOOD Service!!!',
'poor support, not happy ', 'GREAT experience! will come again.', 'okay okay...', ' not BAD', 'Excellent care, excellent staff!', 'good
food and good ambience!', 'Poor response and poor handling of issue', 'Satisfied. But could be better.', 'Good support... quick servic
e.', 'Very good support', 'Average service', '2', 'Very poor', 'Good Support', 'Very Good Service!!!', 'Good Service!', 'Average Servic
e'], 'Rating': [5, 2, 5, 3, 2, 5, 4, 1, 3, 4, 5, 3, 2, 1, 4, 5, 4, 3]}
```


Each list (`S_No`, `Name`, `Feedback`, `Rating`) contains one entry per customer.

The length tells us how many total feedback records exist, including newly added ones.

□ Step 3: Text Cleaning:

Clean all feedback entries as follows:

```
#Step 3: Text Cleaning

cleaned_feedbacks = []

for feedback in feedback_data['Feedback']:

    feedback = feedback.replace('.', '')
    feedback = feedback.replace(',', '')
    feedback = feedback.replace('!', '')
    feedback = feedback.replace('?', '')

    feedback = feedback.strip()

    feedback = feedback.lower()

    feedback = ' '.join(feedback.split())

    cleaned_feedbacks.append(feedback)

feedback_data['Feedback'] = cleaned_feedbacks
```

Activate Windows
Go to Settings to activate Windows.

```
feedback_data['Feedback'] = cleaned_feedbacks

for feedback in feedback_data['Feedback']:
    print(feedback)
```

```
very good service
poor support not happy
great experience will come again
okay okay
not bad
excellent care excellent staff
good food and good ambience
poor response and poor handling of issue
satisfied but could be better
good support quick service
very good support
average service
2
very poor
good support
very good service
good service
```

Activate Windows
Go to Settings to activate Windows.

1.Remove Leading and Trailing Spaces: Use `strip()`.

2.Cleans all feedback text by removing punctuation (. , ! ?) trimming spaces, converting text to lowercase Use `lower()`, and replacing multiple spaces with a single space, making the data ready for analysis.

□ Step 4: Word Count Insights:

Creates a function that counts how many feedback entries contain a given word and is used to find the occurrences of "**good**", "**poor**", and "**excellent**" in the feedback dataset.

```
# Step 4: Word Count Insights

def count_word_in_feedbacks(word):
    count = 0

    for feedback in feedback_data['Feedback']:
        if word.lower() in feedback.lower().split():
            count += 1

    return count

#Use this function to print:
print("Feedbacks containing 'good':", count_word_in_feedbacks("good"))

print("Feedbacks containing 'poor':", count_word_in_feedbacks("poor"))

print("Feedbacks containing 'excellent':", count_word_in_feedbacks("excellent"))
```

```
Feedbacks containing 'good': 7
Feedbacks containing 'poor': 3
Feedbacks containing 'excellent': 1
```

Activate Windows
Go to Settings to activate Windows.

□ Step 5: Final Summary & Insights:

5.(1).Display the final cleaned feedback_data (dictionary of lists)

```
# Step 5: Final Summary & Insights

print("Final Cleaned Feedback Data:")
print(feedback_data)
```

```
Final Cleaned Feedback Data:
{'S_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18], 'Name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Raj', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha', 'john', 'Divi', 'Mathi', 'Zara', 'Shri', 'Shri', 'Mathi', 'Zara'], 'Feedback': ['very good service', 'poor support not happy', 'great experience will come again', 'okay okay', 'not bad', 'excellent care excellent staff', 'good food and good ambience', 'poor response and poor handling of issue', 'satisfied but could be better', 'good support quick service', 'very good support', 'average service', '2', 'very poor', 'good support', 'very good service', 'good service', 'average service'], 'Rating': [5, 2, 5, 3, 2, 5, 4, 1, 3, 4, 5, 3, 2, 1, 4, 5, 4, 3]}
```

5.(2).Print the average rating from all feedback

```
# Print the average rating from all feedbacks.

average_rating = sum(feedback_data['Rating']) / len(feedback_data['Rating'])

print("Average Rating:", round(average_rating, 2))
```

```
Average Rating: 3.39
```

```
sum(feedback_data['Rating'])
```

```
61
```

```
len(feedback_data['Rating'])
```

```
18
```

average_rating = sum(feedback_data['Rating']) / len(feedback_data['Rating'])

average_rating = 61/18 = 3.39

5.(3).Find and display the feedback with the longest comment (in terms of word count)

- The feedback contains 7 words.

```
# display the feedback with the longest comment

longest_feedback = max(
    feedback_data['Feedback'],
    key=lambda x: len(x.split())
)

print("Longest Feedback:")
print(longest_feedback)
```

```
Longest Feedback:
poor response and poor handling of issue
```

5.(4).Print the list of unique words used across all feedbacks (avoid duplicates)

- `set()` automatically removes duplicates.
- `sorted()` displays words alphabetically.

```
# Print the list of unique words

unique_words = set()

for feedback in feedback_data['Feedback']:
    words = feedback.split()

    for word in words:
        unique_words.add(word)

print("Unique Words:")
print(sorted(unique_words))
```

```
Unique Words:
['2', 'again', 'ambience', 'and', 'average', 'bad', 'be', 'better', 'but', 'care', 'come', 'could', 'excellent', 'experience', 'food',
'good', 'great', 'handling', 'happy', 'issue', 'not', 'of', 'okay', 'poor', 'quick', 'response', 'satisfied', 'service', 'staff', 'supp
ort', 'very', 'will']
```

Activate Windows
Go to Settings to activate Windows.

- Displays the final cleaned dataset, calculates the average customer rating, identifies the longest feedback based on word count, extracts all unique words from the feedbacks, and optionally sorts all feedback records by rating in descending order.

□ (Optional): Sort feedbacks by rating (highest to lowest) using zip() and sorted(). Display the sorted entries.

- Using `zip()` and `sorted()`

```
# Sort feedbacks by rating (highest to lowest)

combined_data = zip(
    feedback_data['S_No'],
    feedback_data['Name'],
    feedback_data['Feedback'],
    feedback_data['Rating']
)

sorted_feedbacks = sorted(
    combined_data,
    key=lambda x: x[3],
    reverse=True
)

print("Feedbacks Sorted by Rating:")

for record in sorted_feedbacks:
    print(record)
```

Activate Windows
Go to Settings to activate Windows.

```
Feedbacks Sorted by Rating:
(1, 'Ravi', 'very good service', 5)
(3, 'Sam', 'great experience will come again', 5)
(6, 'Divya', 'excellent care excellent staff', 5)
(11, 'john', 'very good support', 5)
(16, 'Shri', 'very good service', 5)
(7, 'Arjun', 'good food and good ambience', 4)
(10, 'Nisha', 'good support quick service', 4)
(15, 'Shri', 'good support', 4)
(17, 'Mathi', 'good service', 4)
(4, 'Anu', 'okay okay', 3)
(9, 'Leela', 'satisfied but could be better', 3)
(12, 'Divi', 'average service', 3)
(18, 'Zara', 'average service', 3)
(2, 'Meera', 'poor support not happy', 2)
(5, 'Raj', 'not bad', 2)
(13, 'Mathi', '2', 2)
(8, 'Kiran', 'poor response and poor handling of issue', 1)
(14, 'Zara', 'very poor', 1)
```

Activate Windows